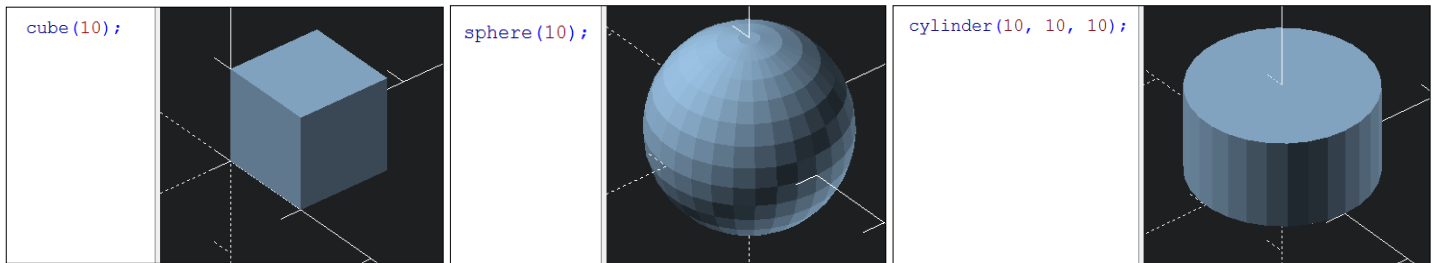


Eastside Preparatory School

OpenSCAD Tutorial

3D SHAPES

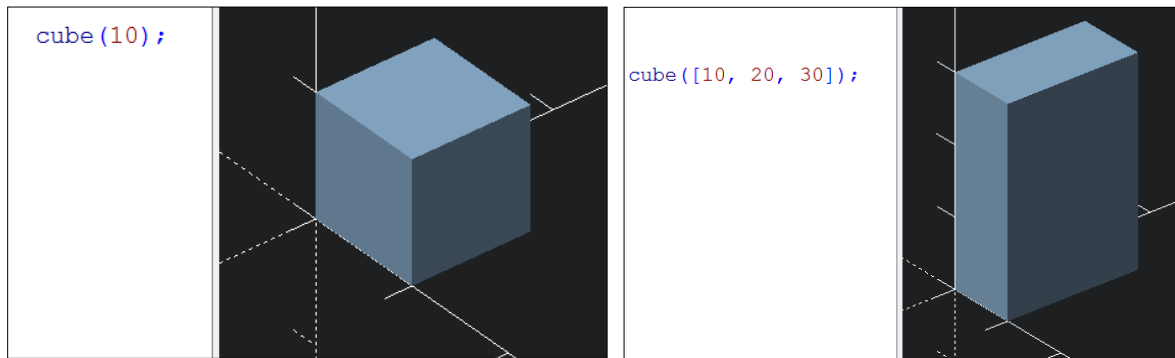
To make something on OpenSCAD, you have to start with one of three basic shapes: a cube, a sphere, or a cylinder. (Or you can make your own completely new shape, explained later.)



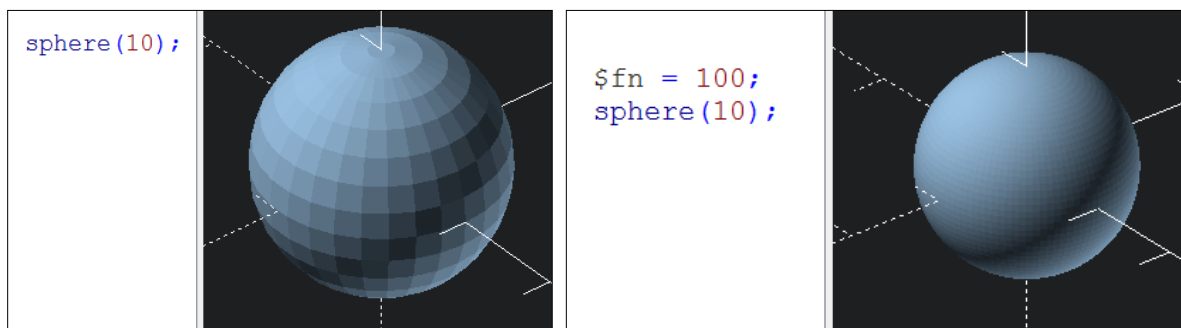
Each shape has parameters that you need to fill in.

A cube can have one parameter (for an equilateral cube), or multiple (for the cube to have different values in the x, y, and z dimensions).

For one parameter, you'll only need one set of grouping symbols: (value), but for multiple parameters, you'll need two sets of grouping symbols: ([x, y, z]). The [x,y,z] list is often referred to as a vector in openSCAD documentation. This is true for cubes and transformations. ('Tis unnecessary for cylinders.)

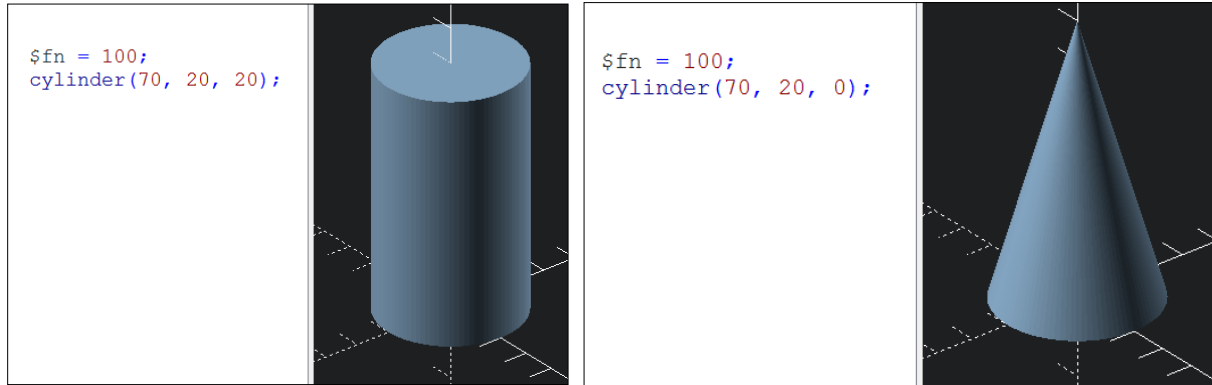


Spheres only have one parameter, being their radius. As you probably notice, the sphere (and the cylinder) has many corners and flat surfaces, which isn't how a sphere should be. To give it better resolution, type **\$fn = 100;** (default is 10) at the beginning of the document. Doing this does give it better resolution, but then also takes longer for your computer to render. The \$fn variable can also be set on individual shapes by adding it to the argument list. i.e. `sphere(4, $fn=1000);`

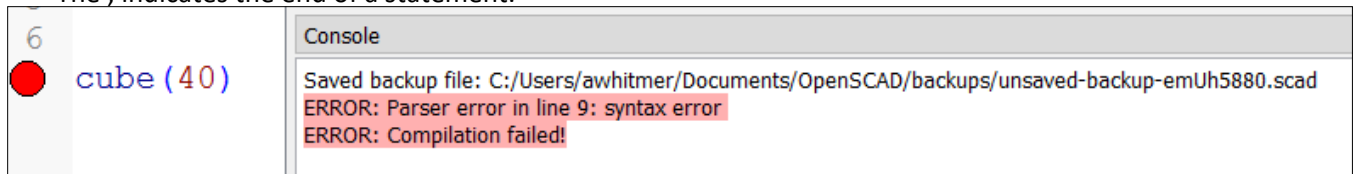


Eastside Preparatory School

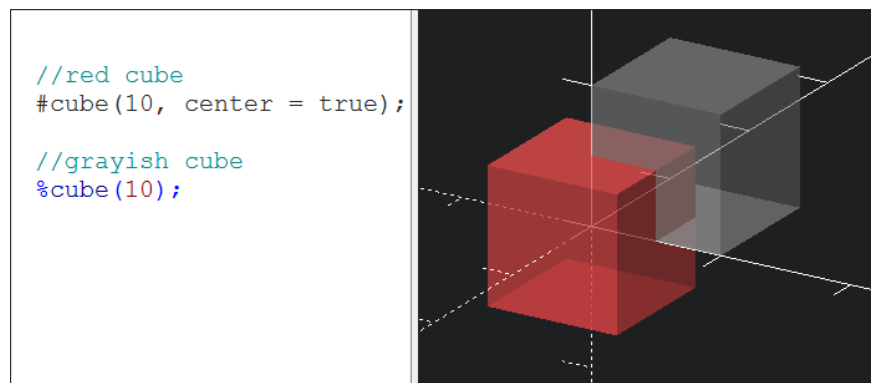
Cylinders have three values; (height, bottom radius, top radius). Cylinders only require one set of grouping symbols. To make a cone, just set one of the radii to zero.



Every shape must be followed by a semicolon, e.g.: `cube(40);` or else the program will get mad at you. The `;` indicates the end of a statement.



To center a shape on the origin, you can tack on **`center = true`** after the numbers and an additional comma, and inside the parenthesis. E.g.: **`cube([10, 20, 30], center = true);`** or **`cube(10, center = true);`** Spheres are automatically centered on the origin, cylinders are centered on the positive z axis, and cubes sit on the positive x-y plane with a corner on the origin.



(The `#` and `%` are just there for color. Red is the centered cube (`#`) and grayish is the (`%`) otherwise they will be the default.)

To comment things out, just use `//blah` or `/* blah */` as you normally would in other coding languages.

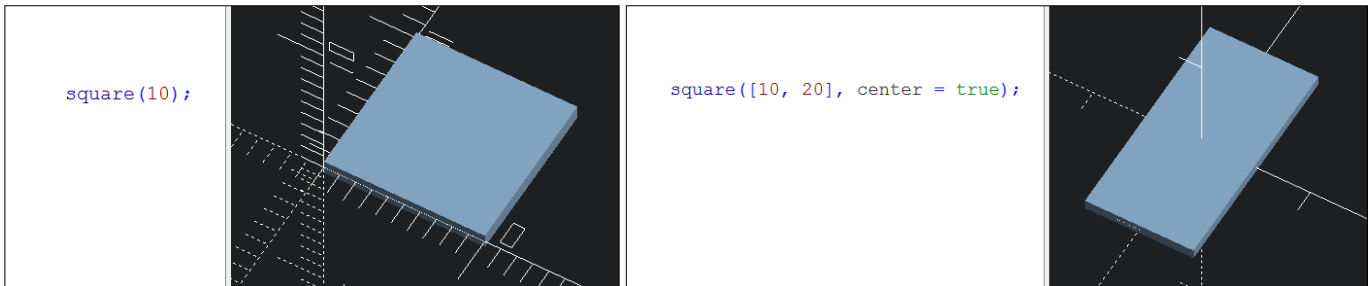
Eastside Preparatory School

2D SHAPES

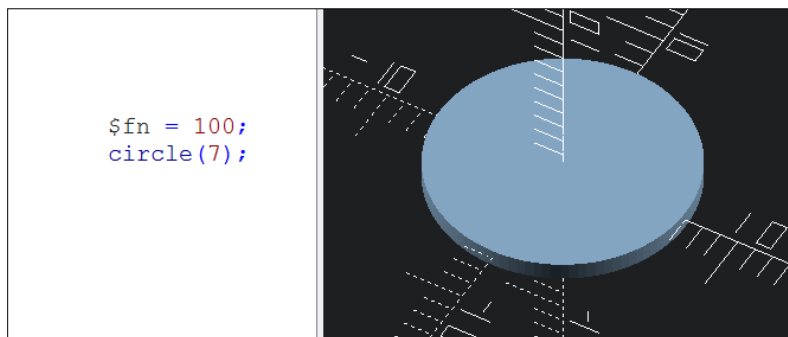
Actually, 2D shapes can be quite useful for 3D modeling. Especially for fun things like extrusions.

There are two basic 2D shapes: a square, and a circle. Except for lacking a third dimension, they are exactly the same as their 3D counterparts.

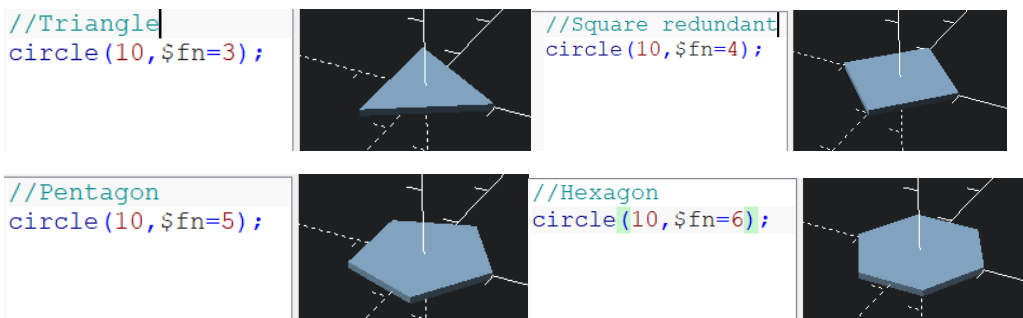
Again, a square can have one parameter (to make it a real square) or two (to make it rectangular). One parameter calls for one set of parenthesis, and two parameters calls for two sets of parenthesis.



Circles are practically the same. One parameter: their radius. \$fn = 100; gives them better resolution.



Don't be fooled! These shapes do look like they have some thickness to them, but theoretically they are infinitely thin and therefore can't be 3D printed. OpenSCAD wouldn't be able to show them to you otherwise. Note that circle in conjunction with \$fn gives you all the regular polygons.



Eastside Preparatory School

MORE COMPLICATED SHAPES

To make your own shapes, you have to use either the `polygon()` function (for 2D shapes) or the `polyhedron()` function (for 3D shapes). These shapes are more complex, as you're basically plotting the points of each shape on a 2D or 3D grid, and telling the program how to connect the dots.

POLYGON

A polygon function has three parameters, but you can forget about the third one ("convexity") because it really doesn't do much, it only helps preview mode do a better job of rendering your shape.

The first parameter is a list of points (as many as you want), and the second is a list of "paths" (as many as are needed) to connect the points. The order in which you connect them will matter in more complex shapes.

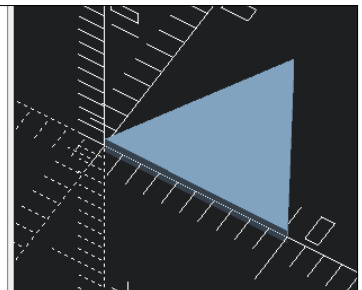
They are written like so:

```
polygon (points = [ [x1, y1], [x2, y2], [x3, y3] ], paths = [ [0, 1, 2] ] );
```

The paths are a list of numbers, telling the program to connect point 0 to point 1 to point 2 and so on.

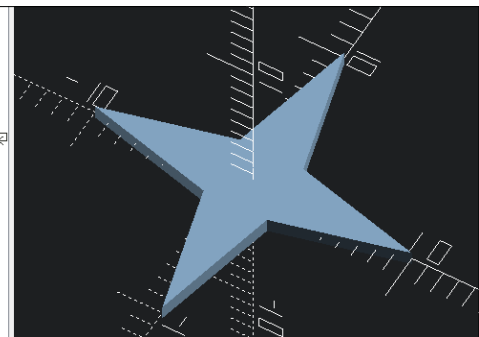
Because this is a programming language, you're going to **start counting from 0** (the first point listed).

```
polygon(points = [[0,0], [5, 8.6], [10,0]], paths = [[0, 1, 2]]);
```



For a more complicated shape, you're gonna need more points, and more paths.

```
polygon(points = [[10,0], [2,2], [0,10], [-2,2], [-10,0], [-2,-2],  
[0,-10], [2,-2]], paths = [[0, 1, 2, 3, 4, 5, 6, 7]]);
```



There is a lot of power in the polygon tool, most of the libraries that create gears and screws use mathematical functions to generate the lists of points and paths and then feed them to the polygon function.

Eastside Preparatory School

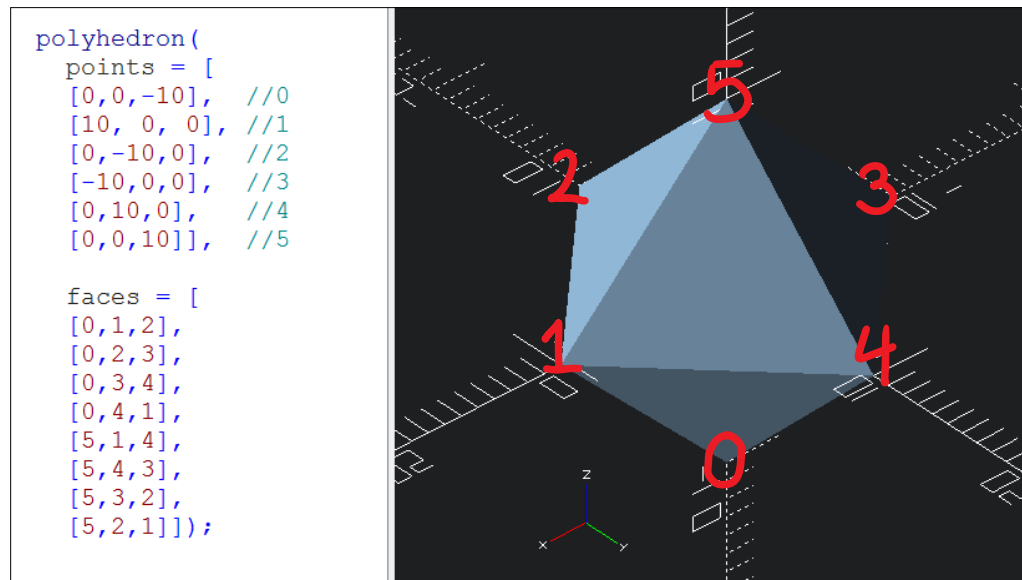
POLYHEDRON

Polyhedrons are more complicated because now, you can't just connect the dots to make edges. Instead, you have to connect the dots to make faces. These paths are harder to make. **Each of these faces can have three or four sides at most.**

Pretend I want to make the octahedral diamond-like shape on the right.

It has six points and eight faces. This is how it would be coded:

```
polyhedron ( points = [ [0, 0, -10], [10, 0, 0], [0, -10, 0], [-10, 0, 0], [0, 10, 0], [0, 0, 10] ],  
             faces = [ [0, 1, 2], [0, 2, 3], [0, 3, 4], [0, 4, 1], [5, 1, 4], [5, 4, 3], [5, 3, 2], [5, 2, 1] ] );
```



First I list the points, and then I connect them in triangles.

The order in which they are connected matters. For this example, I connected them all in a counter-clockwise pattern.

E.g.: Point 0 to 4 to 1 are connected in a counter clockwise fashion. 1-0-4 or 4-1-0 would have done exactly the same thing.

When making points on the opposite side, like 5-3-2, keep in mind that the order you connect them in has to be the same when looking at the face from the outside, not from the perspective you may currently see them at.

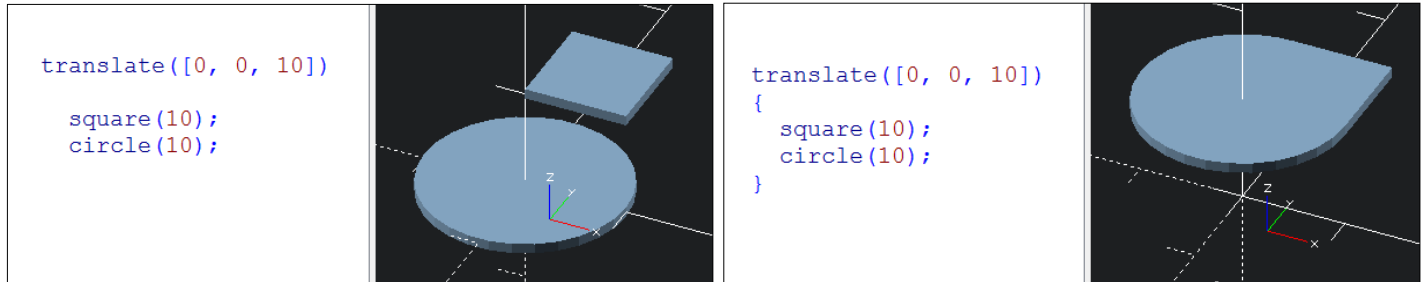
It doesn't matter what order you make the faces, as long as the points are connected in the same way. The reason for this is that the model needs to have a sense of what the inside and outside of your model is in order for it to later be printable. When looking at the face straight on, all the points need to go in the same direction, counter-clockwise in this example.

Eastside Preparatory School

TRANSFORMATIONS

Now that you have some shapes, you're probably going to want to move them around and do things to them. These would be transformations.

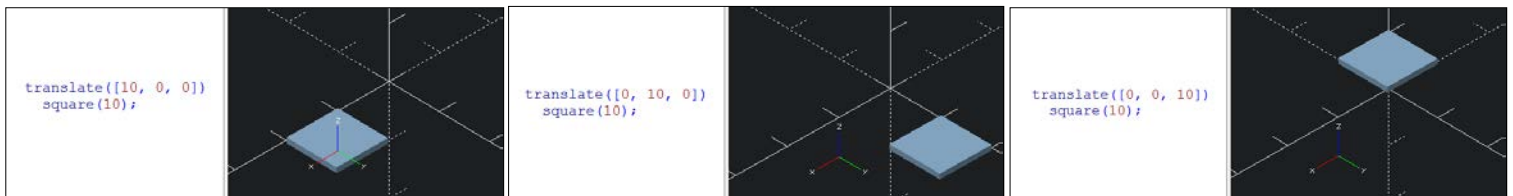
When applying a transformation to a shape, **the transformation will come before the shape. The transformation only applies to the shape directly after it.** If the translation applies to multiple shapes, the shapes will need to be in curly brackets: { }. We say the translation operates on the shape, because of this, there is no semi-colon as the translation is modifying the shape. The shape still has a semi-colon to execute the command.



TRANSLATION

The translate transformation is written like so: **translate ([x, y, z])** followed by a shape.

It has an x, y, and z value. Each value is how many scad units (typically thought of as millimeters) the shape will be shifted over in that direction.

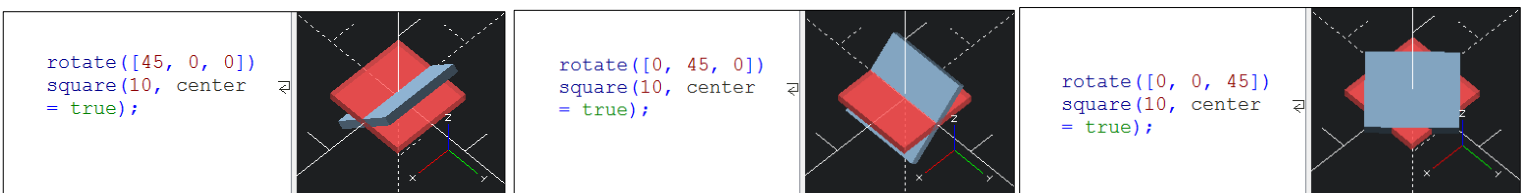


(10 unit translation in the x, y and z directions.)

ROTATION

Similar to translations, rotation is written like this: **rotate ([x, y, z])** followed by a shape.

The x, y, and z values are the number of degrees the shape is rotated around each axis (clockwise).



(45* rotation around the x, y and z axis. Red square is unaffected.)

Eastside Preparatory School

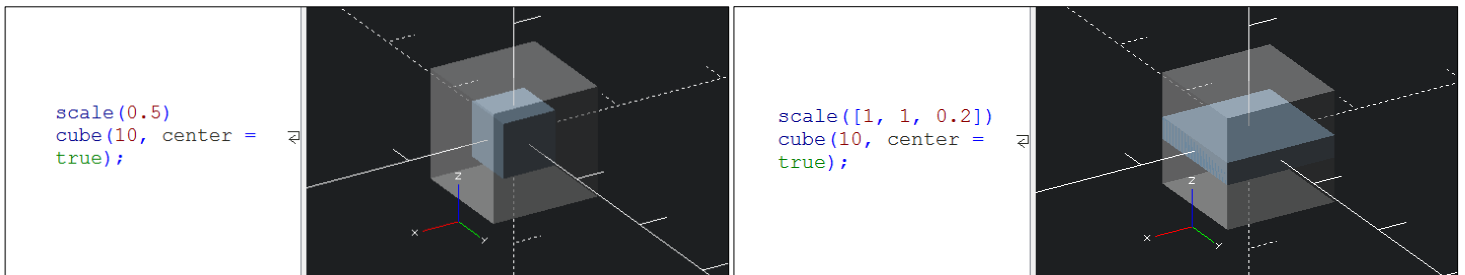
SCALING

Shapes can be scaled up or down with the scale function. Typically, you will want to handle as much as possible with the shape primitives and scale as a last resort.

The x, y, and z values are multipliers for the shape's size in each dimension. A value of 1 won't change the shape. A value greater than one will enlarge the shape, and a value less than one will shrink the shape.

For a uniform scale (same in each direction) use **scale (value) [shape]**

For non-uniform scaling (difference in each direction) use **scale ([x, y, z]) [shape]**

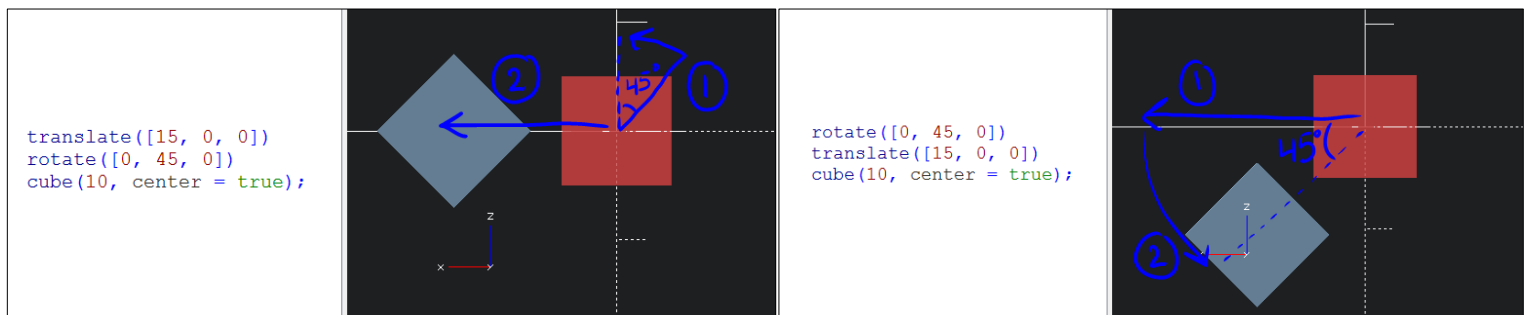


(The see-through cube is unaffected, blue is scaled.)

STACKING TRANSFORMATIONS / OPERATIONS

Of course, you can do multiple transformations to the same shape (or set of shapes), but **the order of the transformations matter**.

The code is executed from the inside out, starting with the shape, and works its way up through the transformations applied to it, then to the next shape and so on.



For example, rotating THEN translating gives a different result than translating THEN rotating. This is because **rotations always happen around the axis**, not around the center of the shape. A common practice is to handle your rotation first on a shape with center=true and then translating it to where it needs to be (i.e. the example on the left).

Eastside Preparatory School

OPERATIONS

Operations make things even more fun. There are three main operations; union, difference, and intersection.

DIFFERENCE

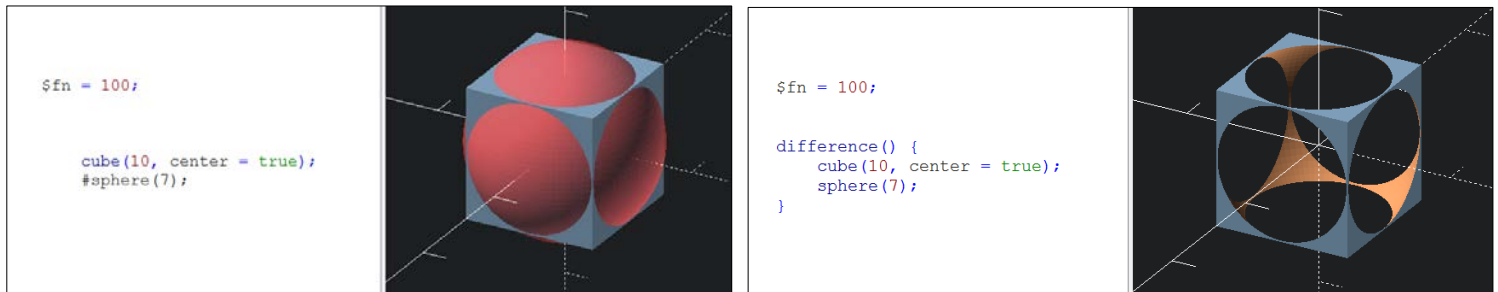
Difference is the operation that subtracts one shape from another to get even more interesting shapes.

Just like regular subtraction, it takes the top shape listed and subtracts the shape underneath (or next; spacing doesn't matter). Curly brackets are required to group the shapes together.

The function is written like this: **difference() { shape#1(n); shape#2(n); }**

Of course, you can apply other transformations to these shapes as well:

difference() { translate([0, 0, 10]) shape#1(n); rotate([0, 0, 90]) shape#2(n, center = true); }

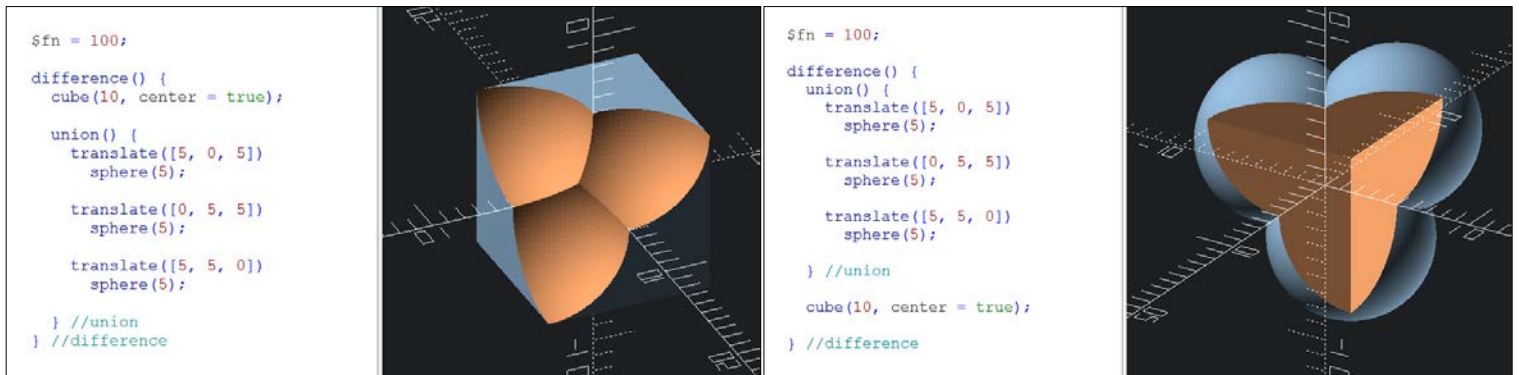


(Sphere overlapping cube vs. sphere subtracted from cube)

UNION

Union simply joins multiple shapes together. There is no visible difference between unioned and ununioned shapes however they need to be unioned to print. If no operator exists on a scad file, the engine wraps your entire file in a union(){ .. }

The most useful application of the union function is when you need a function to apply to more than two shapes. For example, subtracting a bunch of shapes from one object, or vice versa.



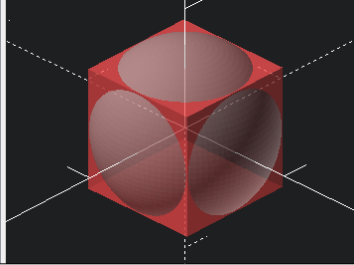
With the union function, multiple shapes can be treated like one. This also can help fixing shapes that don't scale correctly, this is similar to grouping shapes in programs like powerpoint and inkscape.

Eastside Preparatory School

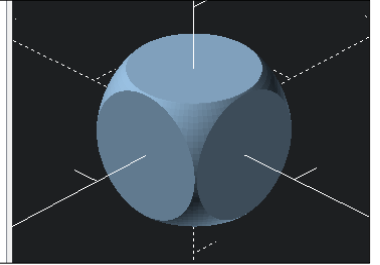
INTERSECTION

Like all of these, as the name suggests, this function takes the intersection between multiple shapes. For it to work, **the shapes must be overlapping**. If there is no overlap between the shapes, there will be nothing to display. This function can apply to as many shapes as you want, but they will need to be in curly brackets.

```
$fn = 100;  
  
#cube(10, center = true);  
%sphere(7);
```



```
$fn = 100;  
  
intersection() {  
  cube(10, center = true);  
  sphere(7);  
}
```



(Overlapping cube and sphere vs. their intersection)

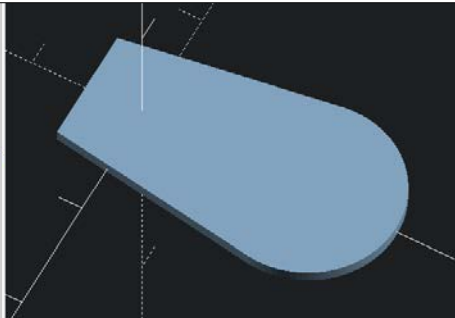
MORE ADVANCED OPPERATIONS

These make everything way better.

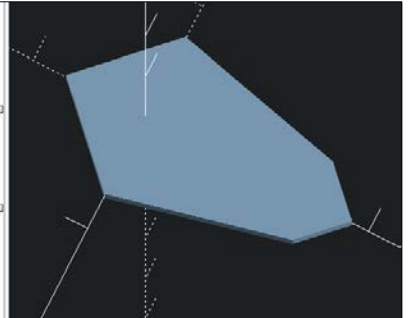
HULL

The hull function fills in the space between shapes (think the shape plastic wrap would make if stretched around the shapes), as long as they have the same number of dimensions. You can use it to stretch shapes into something else that is more interesting. Sadly, **you can't fill in the space between a 2D and 3D shape**. That would just be confusing.

```
$fn = 100;  
  
hull() {  
  square(10, center = true);  
  translate([0,15,0])  
  circle(8);  
}
```



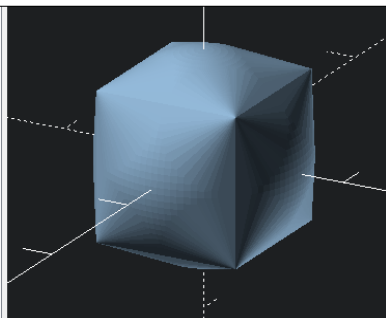
```
hull() {  
  rotate([0,0,45])  
  square(10, center = true);  
  
  translate([0,15,0])  
  rotate([0,0,45])  
  square(5, center = true);  
}
```



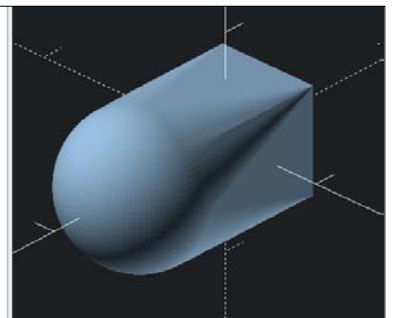
(2D hulled shapes)

With the hull function, you can get some strange baloney-looking shapes.

```
$fn = 100;  
  
hull() {  
  cube(10, center = true);  
  sphere(7);  
}
```



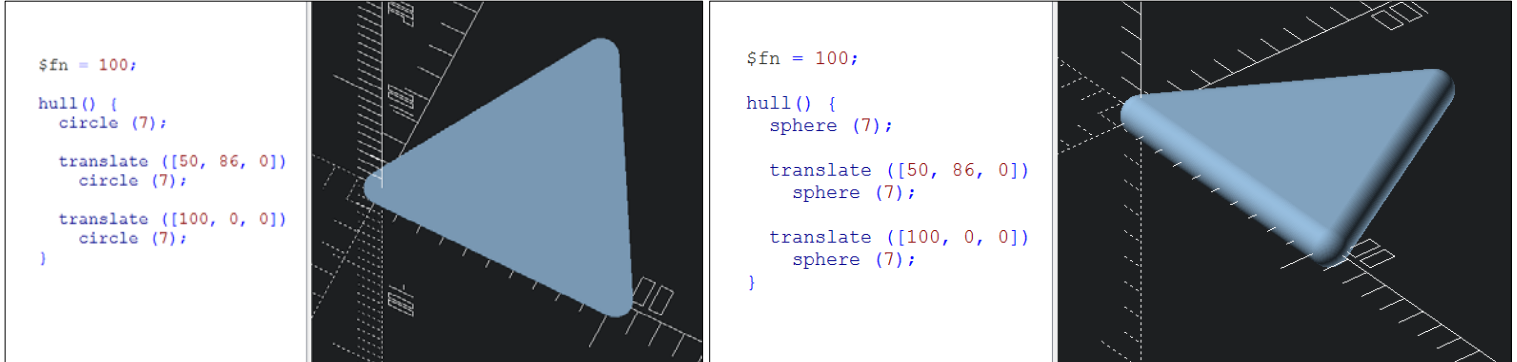
```
$fn = 100;  
  
hull() {  
  cube(10, center = true);  
  translate([10,0,0])  
  sphere(7);  
}
```



Eastside Preparatory School

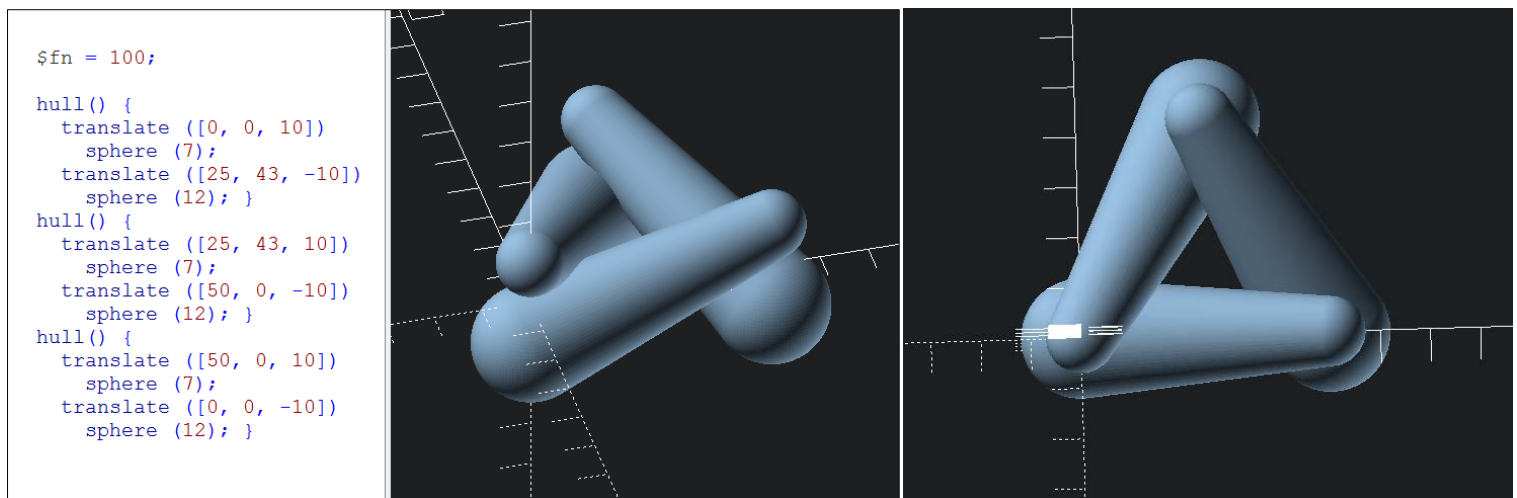
(3D hulls of a cube and a sphere)

The function also works with more than two shapes.



(2D and 3D hulls of three circles and spheres)

It can also be used to make some pretty neat shapes from multiple hulls:



(Different views of the same shape)

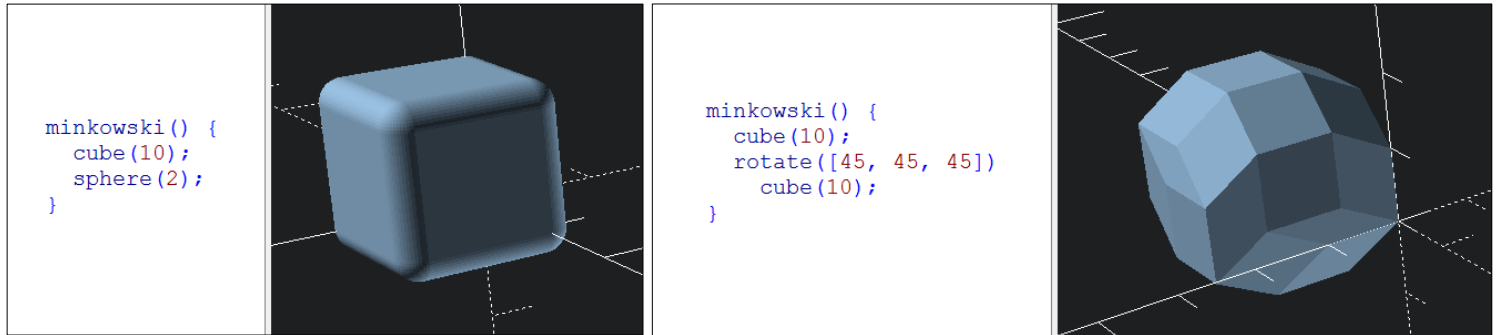
MINKOWSKI

Minkowski was a famous mathematician who dealt with transformations of relatively. It doesn't really make much sense why his name is used for this function, but that's just how it is.

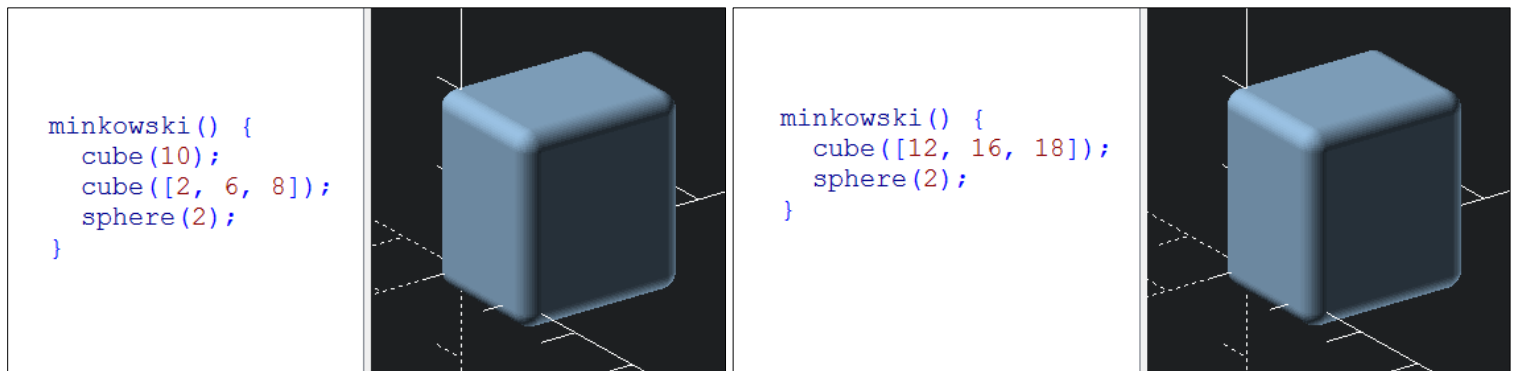
The Minkowski function places a shape on every point of the entire surface of another shape to get a nifty fusion of the both of them. It is especially good for rounding the outside of shapes. If your computer is particularly slow, it may take a while to render.

Eastside Preparatory School

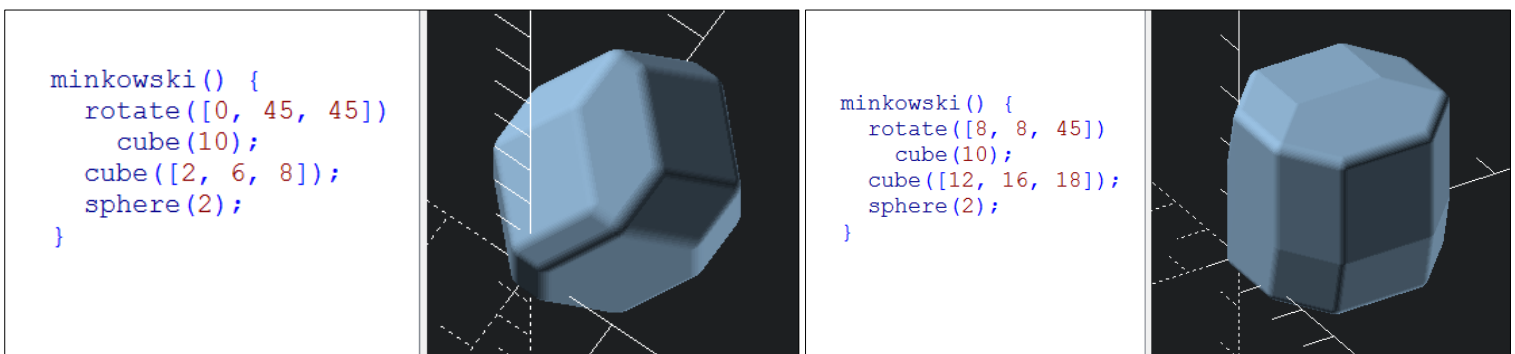
Also, believe it or not, you get exactly the same shape if you roll a sphere around a cube as when you roll a cube around a sphere. **The order of shapes doesn't matter.** Also, this function can be applied to as many shapes as you want, though most (if not all) final shapes can simply be made from two.



There are many nifty things that can be done with the Minkowski function.



Functions with more than one of the same shape can be made exactly the same with only one. E.g.: more than one cube (oriented in the same way) is unnecessary.

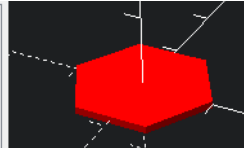


On the other hand, if you rotate one of the cubes, you get a completely different result that can't be replicated with only two shapes.

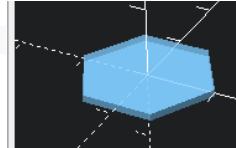
Eastside Preparatory School

While not relevant for 3D printing, it can be helpful to color objects for clarity. Like other operators, the color operator will act on the subsequent shape. There are a variety of named colors and you can pass a vector of [red,green,blue,alpha] where each value is between 0 and 1 which indicates which percentage of red, green, blue and transparency is to be applied.

```
//Hexagon  
color("red")  
circle(10,$fn=6);
```



```
//Hexagon  
color([0.5,0.8,1,0.75])  
circle(10,$fn=6);
```



DEBUGGING

Often it is helpful to know the numerical output of something. In those cases, the echo(); command will print text or a value to the console.

```
a_variable = 2;  
b_variable = a_variable + 4;  
echo(b_variable);
```

```
Compiling d  
ECHO: 6  
Compiling d  
Geometries  
Geometry c
```

EXTRUSIONS

Extrusions come in two flavors in openSCAD, linear_extrude and rotate_extrude. At this point it would be useful to talk about how to find out how to do new things on openSCAD. If you head to https://en.wikibooks.org/wiki/OpenSCAD_User_Manual/ (linked from openscad.org)

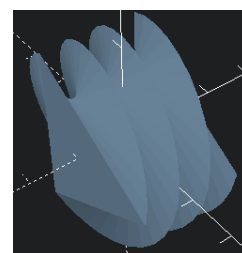
One of the pages is https://en.wikibooks.org/wiki/OpenSCAD_User_Manual/2D_to_3D_Extrusion

```
linear_extrude(height = fanwidth, center = true, convexity = 10, twist = -  
fanrot, slices = 20, scale = 1.0)
```

Along with some explanatory text, it shows you how the linear_extrude action is constructed. You can execute linear_extrude(10,true,10,90,100,0.8) or you can explicitly set the named variables linear_extrude(height=10,twist=90). Most actions have sensible defaults if you don't want to figure out what things like convexity mean (just leave it unset).

Height defines how tall the object will be in the Z direction or both Z and -Z directions if center=true is set. Twist defines how many degrees it rotates around the Z axis. Slice defines how steps are taken, essentially how smooth the object is when constructed. Scale determines how the shape grows or shrinks over the course of the extrusion. In general, you will be extruding objects then rotating them and finally translating them. Note that this means that your code will look like:

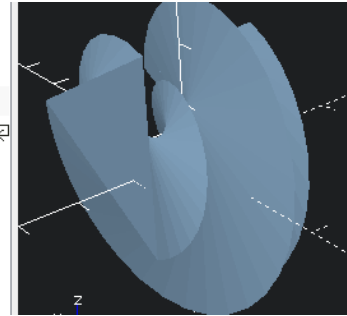
```
translate([0,10,0]){  
    rotate([90,0,0]) {  
        linear_extrude(height=20,twist=360){  
            circle(r=20, $fn=3); //builds a triangle  
        }  
    }  
}
```



Eastside Preparatory School

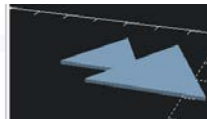
Since `linear_extrude` twists around the axis, offsetting your shape first from that axis can give you different results.

```
]translate([0,10,0]){
  rotate([90,0,0]) {
    linear_extrude(height=20,twist=360){
      translate([5,5,0])
      circle(r=20, $fn=3); //builds a
    triangle
  }
}
```

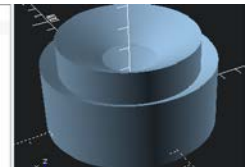


`rotate_extrude` is slightly more complex. It works in the following way. It will rotate your object around the Y axis and then sweep that resulting object around the z axis (360 degrees currently, likely settable to arbitrary degrees in the next version of openSCAD).

```
translate([-30,10,0])
union(){
  circle(r=18, $fn=3); //triangle
  translate([5,15,0]){
    circle(r=13,$fn=3);
  }
}
```



```
rotate_extrude(convexity=10, $fn=360)
{translate([-30,10,0])
  union(){
    circle(r=18, $fn=3); //triangle
    translate([5,15,0]){
      circle(r=13,$fn=3);
    }
  }
}
```



TEXT

Text can be entered via `text("text you want rendered")`; The text function is fully configurable and even supports arbitrary ttf fonts. Documentation is here:

https://en.wikibooks.org/wiki/OpenSCAD_User_Manual/Text. Note that the function just draws the 2D version, you then need to extrude the font into a 3D shape, typically using `linear extrude`. This apparent extra step gives you lots of flexibility.

```
text("NCCE");
```



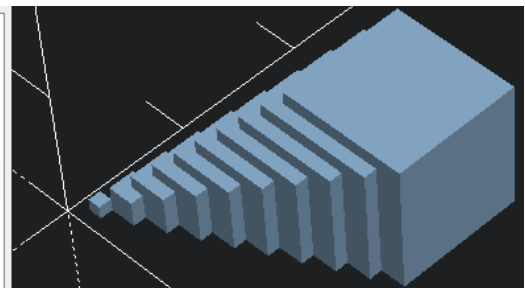
```
linear_extrude(height=10,twist=30)
{text("NCCE");
```



LOOPS

Loops exist to reduce the amount of code you write which in turn makes your code easier to change. They are in the following format:

```
1 union (){
2   for (i=[1:10]){
3     translate ([i,i,0])
4     cube ([i,i,i]);
5   }
6 }
```



Eastside Preparatory School

Using `i=[start:end]` will create an instance for each trip around the loop at every value between the two points (inclusive) incrementing by 1. Using `i=[1,2,6]` will do one loop for each of the listed values. You can also create more complex sequences using functions and hand them to loops.

MODULES

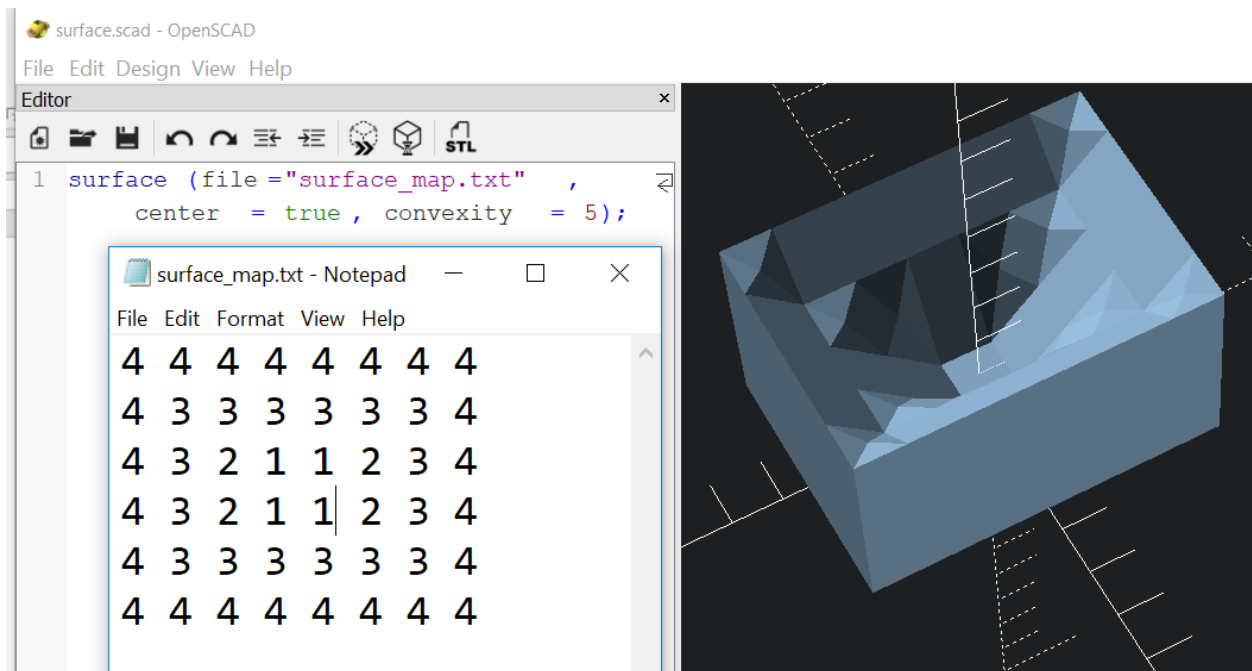
Modules allow you to group primitive shapes into logical units. They should be used a lot in order to keep complex shapes organized. Once you get used to the idea of generating shapes in openSCAD they are your next stop. Once you build a module, you can then use that module name to generate that shape in its entirety. Most libraries of shapes in the openSCAD world are modules such as `gear()` or `screw()` and they accept a wide range of parameters to tweak the over-arching design. As you get more practice using openSCAD you will develop primarily through mixing modules together.

FUNCTIONS

Functions exist in the mathematical sense. The expectations are that you give a function some parameters and you receive some parameters back. It could be a number or a list of points that will later be used in a polygon. Functions do not create shapes, they transform a set of inputs into a set of outputs. Typically they are used in conjunction with modules but can also be more generic like generating the Fibonacci series. You might use them for something like creating a bunch of cubes who's heights follow the Fibonacci series or to calculate complex curves involved in making gears.

SURFACE MAPS

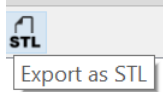
You can also use external files to create surface maps by giving height parameters in a matrix. Additionally, you can import any STL or DXF file into openSCAD and manipulate it.

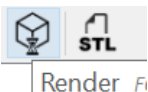


Eastside Preparatory School

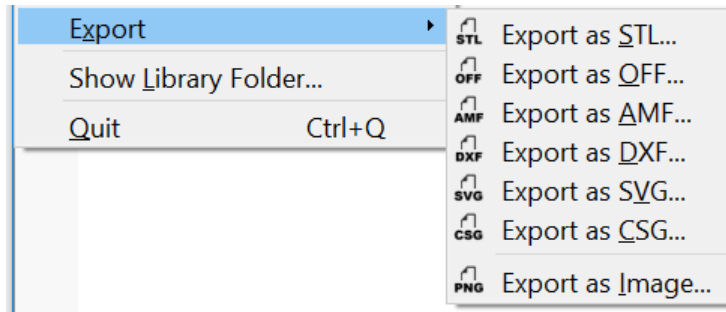
3D PRINTING

3D printing programs receive an STL file as input. You can generate that file via the STL export button:



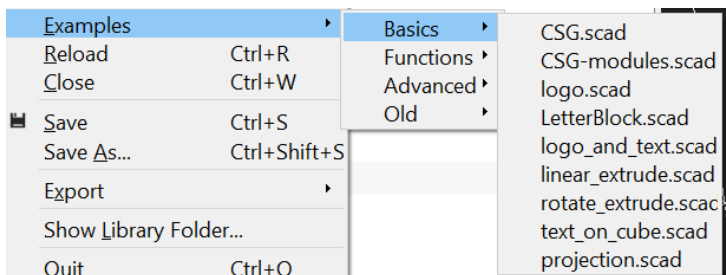
this typically requires you to do a full render as part of the process . This can take some time if you have a complex shape or are using complex functions like the minkowski function.

If you need some other format, there are a variety that are available including flat image files:



LEARNING MORE

https://en.wikibooks.org/wiki/OpenSCAD_User_Manual has plenty of documentation if you want to get some overview of the whole system. It is a good place to start if you are already familiar with programming concepts. Alternatively the examples file is a good place to see documented models.



Additionally, many of the files on thingiverse are made with openSCAD and you can download those model files as well. They vary widely in terms of coding style and they are a useful starting point for the things that you want to create, our TEDx gears were adapted from one of those files.

